Benchmark Methodology

What the Benchmark Is For

The first benchmark is a prompt-selection study. Its purpose is to choose the translation prompt that will later be used in the main experiment.

To isolate prompt effects:

- one model is used throughout an experiment;
- every selected C program is translated with every candidate prompt;
- all prompts use the same visible tests and Judge cases for that program;
- prompt order is shuffled reproducibly so one prompt is not always called first or last.

The repository currently contains one control prompt, `direct-v1`, and five prompts adapted from research papers. Each run records the exact prompt text, identifier, and SHA-256 hash.

Where the Prompts Came From

The paper-derived prompts were selected from the research-prompt catalog in `Translation Task Prompts in LLM Research Papers.xlsx`. They were adapted only as needed to translate complete C programs into Rust in an unattended benchmark.

Prompt	Origin	Approach retained	Adaptation for OxCidation
`direct-v1`	OxCidation control; not derived from a paper	Direct behavior-preserving translation	Existing project baseline, retained as the control.
`trace-zero-shot-v1`	*TRACE: Evaluating Execution Efficiency of LLM-Based Code Translation*	Zero-shot translation	Replaced generic language placeholders with C and Rust and removed response wrappers.
`trace-performance-v1`	*TRACE: Evaluating Execution Efficiency of LLM-Based Code Translation*	Performance-aware zero-shot translation	Kept efficiency and best-practice instructions while adapting the interface to C and Rust.
`realworld-safe-constraints-v1`	*Towards Translating Real-World Code with LLMs: A Study of Translating to Rust*	Zero-shot translation with explicit constraints	Specialized the source language to C and retained only constraints present in the cataloged prompt.
`rustmap-instruction-v1`	*RustMap: Towards Project-Scale C-to-Rust Migration via Program Analysis and LLM*	Instruction-based C-to-Rust translation	Removed interactive clarification because benchmark programs are complete and runs are unattended.
`safetrans-safe-v1`	*SafeTrans: LLM-assisted Transpilation from C to Rust*	Complete translation into safe Rust	Removed Markdown wrapping while keeping safe-Rust,

			compilation, and code-only requirements.

Few-shot, self-refinement, and repair prompts from these papers were not mixed into the initial-translation comparison because they require examples, reference translations, runtime feedback, or an already faulty Rust program. Those would introduce a different experimental stage or an additional variable. The complete provenance and exclusion notes are maintained in `prompts/benchmark/README.md`.

The Available Program Population

The complete accepted-C extraction contains:

Source	Problems with accepted C	Accepted C programs
AIZU	1,874	161,078
AtCoder	1,391	152,282
Total	**3,265**	**313,360**

Exact CodeNet-to-AOJ mapping is now a required preparation stage. Of the 1,874 accepted-C AIZU problems, 1,832 have descriptions that can participate in exact-hash mapping. The completed mapping found 1,763 exact one-to-one matches. The mapped prefetch retrieved complete system suites for 1,738 of them, containing 33,301 input/output pairs; four retrievals failed and 21 were unavailable. The accepted-C baseline count is a separate filter and must be frozen before sampling. The earlier 1,487-suite corpus used an incorrect numeric-ID mapping and is not part of the sampling frame.

When an integrated Judge is configured, the implementation filters the source population before seeded sampling. A program is structurally eligible only when its problem has metadata limits and a complete, non-empty set of matched Judge input/output files. The accepted-C baseline is still executed during evaluation, so this structural filter must not be described as population-wide baseline validation.

There are many programs for some problems and very few for others. In AIZU, the median problem has 6 accepted C programs, while some have thousands. Treating all 313,360 programs equally would therefore make popular problems much more influential than uncommon ones.

The population builder currently avoids this imbalance by selecting one accepted C program per problem. That produces 3,265 candidate programs. The team has not yet decided whether the definitive experiment will keep this rule or select a small capped number, such as two or three, per problem.

If multiple programs are selected, they must remain grouped by problem during sampling and analysis. They solve the same task and use the same Judge suite, so they are related observations rather than independent problems.

Two Samples with Different Purposes

The study requires two disjoint problem sets.

Prompt-selection set

Every candidate prompt is run on this set. Its results are used to compare prompts and freeze one choice.

Final-evaluation set

This set remains untouched while prompts are compared. After selecting a prompt, only that prompt is run here to estimate final performance.

```
eligible problems
  |
  +-> prompt-selection set -> compare prompts -> choose one
  |
  +-> final-evaluation set  -> run the chosen prompt once
```

The split must happen by problem. If different submissions from the same problem appeared on both sides, the sets would not be meaningfully independent.

What Is Measured for Each Combination

OxCidation preserves the process rather than only a final pass/fail label.

Once per C program

- visible-test generation attempts;
- deterministic candidate rejections and stability checks;
- Validator review per candidate and one optional replacement round for rejected cases;
- frozen suite hash or terminal reason for unavailable visible tests.

Semantic terminal outcomes, such as an inconclusive review, are reusable. Operational generation or review failures stop the benchmark before that program's prompt runs and remain retriable with `--resume`.

Before repair

- initial Rust source;
- translation and compilation status;
- visible-test result;
- initial Judge result;
- initial token and duration data when available.

During repair

- failure that triggered each repair;
- Validator classification, diagnosis, and guidance;
- every intermediate Rust version;
- compilation and visible-test result of each attempt.

After repair

- final Rust source;
- final visible-test result;
- final Judge verdict and per-case counts;
- total repairs, duration, and available token counts.

This separation supports four views of prompt quality:

1. **Initial quality:** what the prompt produces without pipeline assistance.
2. **Final quality:** what remains after the allowed repair process.
3. **Repair dependence:** how often a prompt needs intervention and why.
4. **External failures:** what the Judge finds after visible validation.

Choosing the Best Prompt

The data can support compilation rates, Judge acceptance, WA/TLE/RE/MLE counts, repair frequency, case-level pass counts, duration, and token use. The team must still define which measure is primary and how ties are resolved.

That rule must be written before examining the definitive prompt-selection results. Otherwise, the criterion could be changed after seeing which prompt looks best.

Statistical Interpretation

The design has two important dependencies:

- prompts are paired because each prompt sees the same selected programs;
- multiple programs from one problem are clustered because they share the same task and Judge suite.

The final analysis should therefore:

- compare prompts within the same programs;
- weight problems deliberately rather than allowing submission-rich problems to dominate;
- calculate uncertainty at the problem level when multiple programs are used;
- distinguish number of program runs from number of covered problems;
- report Judge coverage separately from overall benchmark coverage.

Before calculating sample sizes, the team must define the smallest prompt difference that would be meaningful in practice. Seeds, split manifests, exclusions, reserve problems, prompts, model settings, Judge data, and analysis rules should then be frozen before the definitive run.

Human Inspection

Automatic results identify patterns and failures, while linked code and reports allow researchers to inspect why they occurred. The workbook is an analysis and navigation artifact; it does not implement reviewer assignment, blinded review, scoring, or adjudication. Any later manual coding protocol must be defined and stored separately before it is used as research evidence.